PlanetCompute: Toward a Verifiable AI Grid of Idle Browsers and Desktops

Design, First Measurements, and a Simulation at a Million Devices

Milind K. Patil
Syncaissa Systems Inc. • PlanetCompute.ai
syncaissa@outlook.com

October 2026

Abstract

We present **PlanetCompute**, a volunteer-computing system that aims to serve AI inference from idle browsers and desktops, and we report what is built, what is measured and what is only simulated. Built: one job format (**WorkPacks**) whose integer kernels give bit-identical results as WebAssembly in a browser and as native code in a Tauri 2 desktop program, which also runs whole chat and image prompts in sandboxed llama.cpp and stable-diffusion.cpp engines; commit-then-random-audit leases; and a **SherpaHub** that cuts matrix multiplies into packs checked by Freivalds' method, which finished two split jobs of up to 262,144 units with no wrong answer on one operator server, though at only 1.5% of the kernel's speed, limited by round trips; we show that over home links such packs are bounded by upload bandwidth. Measured offline on three NVIDIA GPU types, but not yet in the program: single-step diffusion and teacher-forced LLM audits that cost 3.3–8.9% of the work. Analysed and simulated: we argue for cutting the work rather than the model and size packs by a square-root law; derive the exact collusion probability of quorum validation, under which a naive "first three matching answers" rule accepts 5.8% wrong packs at 20% colluders (6.1% in simulation); show that spare replicas, network-distinct quorums and retroactive invalidation remove these errors in simulation; and simulate the orchestrator at a million devices, where, with 2% ordinary cheaters, no wrong answer reached a consumer uncorrected and a low-cost design cut the list-price running cost from $111.54 to $9.80 an hour at the price of latency. No volunteer outside the operator's servers has yet taken part.

Keywords: volunteer computing, compute grid, WorkPacks, AI inference, competitive dispatch, network-distinct quorum, Freivalds' check, audits, WebAssembly, Tauri, llama.cpp, sabotage tolerance

Executive Summary

Every night, millions of PCs and laptops sit idle for hours, mine and yours included. So why not lend that spare power to each other? With PlanetCompute, your idle machine shares its compute by helping run jobs for others, and you collect PlanetComp points in return. When you need serious power yourself, you draw compute from the network and points are deducted from your balance. Your lifetime contributions are tracked separately, for bragging rights. To keep the ledger consistent and tamper-proof, PlanetComp points are recorded as a token on the Solana blockchain. Imagine one day rendering a full 100-minute movie on the world's spare computers while you sleep (long video is designed, not yet built): no render farm, no huge cloud bill, just a community helping each other. It might take a few weeks to finish, but you pay no cloud bill: only your own electricity, through the points you collected by sharing your own idle time.

Of course, this only works if enough compute is contributed to the community pool. As of this paper's release the pool is empty, so we're seeding it with our own compute to get

things started, though even that may not be enough at first. As more members join and the network becomes self-sufficient, we'll gradually reduce our share and may eventually phase it out completely.

Disclaimer

PlanetComp is a ledger of contributions only and may have no monetary value. Check your country's and local laws: if you cannot hold crypto where you live, refrain from transferring any PlanetComp to your wallet, although it may or may not have value. User discretion is required.

1 Introduction

Harvesting idle cycles is older than the web: Xerox PARC's "worm" programs spread one computation across idle workstations at night in 1982 (Shoch and Hupp, 1982), Condor scheduled jobs onto idle workstations from 1988 (Litzkow et al., 1988), and in 1989 volunteers' computers factored large numbers from pieces sent by email (Lenstra and Manasse, 1990). BOINC (Anderson, 2004, 2020) later showed that millions of volunteers will donate compute, Folding@home (Beberg et al., 2009; Zimmerman et al., 2021) reached exascale in 2020, and commercial successors (Golem (Golem Factory, 2016), Salad (Salad Technologies, 2026)) added payment. Generative AI now creates a new kind of demand: image, video and chat inference whose units are, once cut the right way, largely independent of each other.

Four problems persist. (1) **The host dilemma:** native clients run overnight and reach the GPU but must be built, signed and trusted per platform; browser clients need no install but are throttled and die with the tab. (2) **Code safety:** volunteers must trust each project's native binaries. (3) **Consent:** in-browser mining was overwhelmingly covert (Eskandari et al., 2018; Konoth et al., 2018). (4) **Verification and fair credit:** unchecked, a few percent of cheaters poison a result set (Sarmenta, 2002). PlanetCompute answers with four principles: the **WorkPack is the portable unit** (a signed job with fixed inputs whose result is the same bytes on every host); **two hosts, one format**; **competition with consolation** (the first validated finisher receives the largest credit, other matching finishers consolation credit); and **validation as protocol** (quorum, audits, strikes and credit are specified, not left to client goodwill).

1.1 Contributions

1. A **two-host design** in which the browser and the desktop program compute the same packs with bit-identical results (fixed WebAssembly and native kernels, one binary pack format, leases with commit-and-audit) and the desktop also runs whole AI prompts; a general byte ABI for third-party modules is specified but not yet built (Section 2).
2. **Inference WorkPack design**: a communication argument against splitting models, a square-root law for pack size validated in simulation, and audit verification with an escrow rule (Section 3).
3. **QuickMode/BatchMode** queues and **SherpaHub**, with measured kernels, a fault simulator, a live broker and two live split jobs (Section 4).
4. **Offline GPU measurements** of throughput, cross-GPU reproducibility and audit cost, in PyTorch and vLLM; the audits are not yet in the program (Section 5).
5. **Validation and credit**: the exact quorum-race probability, strikes with retroactive invalidation, and commit-then-random-audit (Section 6).

6. **Competitive dispatch** with a closed-form attack analysis and a hybrid rule that is fast, fair and, with penalties, zero-error in simulation (Section 7).
7. An **orchestrator simulated at a million devices**, with its faults, results and running cost (Section 8).
8. A threat model and privacy tiers (Section 9), and `planetCompute`, one program that contributes a machine and serves models on it, with its first measurements and tests (Section 10).

All code, scripts, raw results and test logs are in the repository (https://github.com/syncaissa/planetCompute.cpp).

2 The PlanetCompute Architecture

Figure 1 shows the system as built: one job format, two hosts. Table 1 separates what runs today from what is specified.

Table 1: Built today and specified.

	Built today	**Specified, not built**
Job kinds	Credit WorkPacks (Monte Carlo samples); int8 matrix maths packs (Split); whole prompts for chat, pictures and video (WholeMode, desktop only)	Any third-party module, fetched by SHA-256 and checked against a signed manifest
Kernels	Fixed: WebAssembly SIMD or JavaScript in the browser, native SSE2 or NEON on the desktop; bit-identical, tested on every build	The byte ABI (`pc_alloc`, `pc_run`, `pc_output`; no imports)
Determinism	Integer kernels and conformance vectors	The determinism profile for any admissible module
Transport	Leases on each device's capacity-check socket; the SherpaHub broker's binary pack messages	`POST /v1/claim` and `POST /v1/units/{id}/result`
Checking	Commit-then-random-audit of credit packs; Freivalds on every maths pack; a share of whole prompts repeated on another computer	Network-distinct quorum and competitive race for every unit (simulated, Sections 6 and 7)
GPU	Browser WebGPU WorkPacks (code and audits tested on the CPU; not yet on a real GPU); the desktop's AI engines	Desktop GPU WorkPacks through wgpu

Specified: the general WorkPack.

Definition 2.1 (WorkPack)**.** A WorkPack is a tuple $U = (id, h_M, x, \pi, \mathcal{K}, w, t_{\mathrm{dl}})$ where $h_M = \mathrm{SHA256}(M)$ identifies module M, x is the input byte string, $\pi \in \{\mathtt{cpu}, \mathtt{gpu}, \mathtt{nn}\}$ the tier, $\mathcal{K}$ the minimum host capabilities, w the benchmark-normalised cost used for credit, and t_{dl} the deadline.

In the specification, modules are fetched by hash and checked against a signed manifest before use, so a tampered CDN can deny service but never substitute code. A module **imports nothing** and exports one memory and three functions, `pc_alloc`(n), `pc_run`(p, n) and `pc_output`$()$; the result is $r = \mathrm{SHA256}(M(x))$, and with no imports a module has no clock, randomness, I/O or network. A module is *admissible* if it validates under WebAssembly 2.0 (W3C WebAssembly

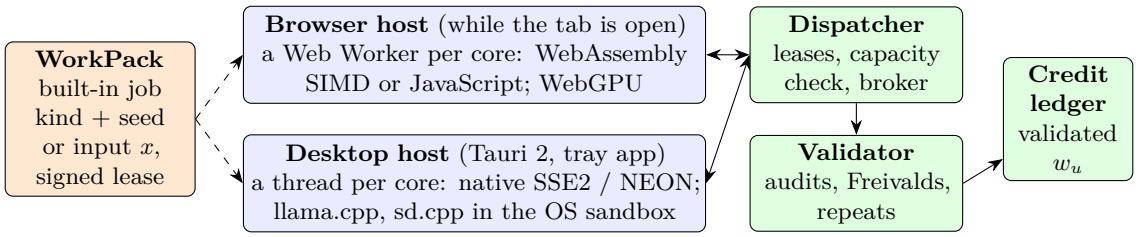


Figure 1: PlanetCompute as built. Both hosts compute the same WorkPack kinds with bit-identical results (WebAssembly or JavaScript in the browser, native kernels on the desktop) and report to one dispatcher, validator and credit ledger. The page comes from one Cloudflare Worker; the program ships as builds signed with the project's Ed25519 key (not yet code-signed by the operating systems) for Windows, macOS and Linux (sd.cpp: stable-diffusion.cpp). Third-party modules fetched by hash, the byte ABI and their `/v1/claim` protocol are specified, not built (Table 1).

Working Group, 2025) with no threads, no relaxed-SIMD and no non-canonical NaN bits in its output; admissible modules give the same bytes in every engine.

Built today. The job kinds are fixed and built in (Table 1), with bit-identical results on both hosts tested on every build, so a Chrome tab and the program on Linux, macOS or Windows form *one* validation pool; the two-verb `/v1` protocol is for third-party modules.

GPU. WebGPU (W3C GPU for the Web Working Group, 2025) runs the browser's GPU WorkPacks; wgpu (gfx-rs contributors, 2026) would run the same WGSL shaders natively on the desktop (planned). The desktop reaches the GPU today through its AI engines, which ship inside the signed program and run in the operating system's sandbox; a job supplies only a model name and inputs. **No tier ever executes native code supplied by a job.**

Politeness. Today a device computes only with recorded consent, on the cores its owner grants and under a CPU cap γ: after each pack of length t a worker rests $t(1-\gamma)/\gamma$, so PlanetCompute's share of a granted core is at most γ, and the program's WorkPacks pause while it answers a chat job or makes a picture. The specification adds user idle, AC power, allowed hours, a thermal limit and other processes' load below 0.20, which a browser observes only in part (Table 16).

3 Designing WorkPacks for Inference

PlanetCompute targets cheap inference for image, music, video, chat and coding. The central design question is how to cut a job into WorkPacks so that the answer arrives with minimum wait.

3.1 Cut the Work, Not the Model

Over the internet the hop between two machines costs far more than the arithmetic it carries. With illustrative values (one-way latency 20 ms, residential upload 2.5 MB/s, an RTX 4090-class GPU), an 8B LLM at 4-bit (32 layers) decodes a token in ≈5 ms on one machine (Table 2).

Diffusion is no different: handing an SDXL latent (131 KB) on after each denoising step costs ≈72 ms against a 0.1–0.3 s step (Table 4), and gains nothing, because step $k{+}1$ needs step k. A job is therefore split only along an axis that is **parallel** and **coarse** (compute per hop at least nine times the hop's cost): requests, seeds, images, tiles, keyframe-anchored video chunks, audio segments, code candidates and test files. Steps, tokens and layers are never split

Table 2: Splitting one LLM across volunteer machines (illustrative).

Split	**Time per token**	**vs. one machine**
Whole model on one machine	≈5 ms	1×
One layer per machine (pipeline)	32×20 ms $= 640$ ms	≈ 130× slower
Every matmul over k machines (tensor parallel)	$2 \times 32 \times 40$ ms $= 2.56$ s	≈ 500× slower

across machines in QuickMode; a model that fits no single volunteer is served as a fixed pipeline lane in the style of Petals (Borzunov et al., 2023). An *inference WorkPack* is thus a run of b independent units of one job, for one *lane* (model hash, quantisation, runtime) that the device already holds in memory.

Frame-parallel video. Video looks sequential but need not be. A plan stage computes a motion plan $m = \Phi(I_0; s)$ with a shared seed s, in which each region moves by closed-form kinematics, $\mathbf{p}_r(t) = \mathbf{p}_r + \mathbf{v}_r t + \frac{1}{2}\mathbf{a}_r t^2$; keyframes $K_j = G_\theta(I_0, m, j\Delta; s)$ are then mutually independent, and in-between frames depend only on their two neighbouring keyframes (Huang et al., 2022). In practice a pack is a keyframe-anchored chunk in the model's native window (16–81 frames, 4–8 frames of overlap), because current video models generate a window jointly. Every job uses at most three stages: a small plan, one wide parallel stage, and a merge (Figure 2).

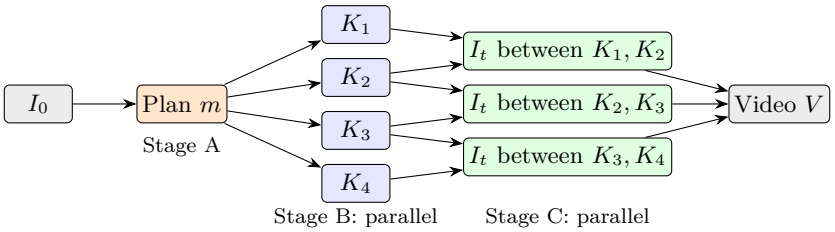


Figure 2: Frame-parallel image-to-video (designed, not yet built). Stage A runs once; Stages B and C fan out to as many idle machines as there are frames. Each box is one or more WorkPacks.

3.2 How Big Should a Pack Be?

Cut a stage of W units into packs of b, pulled by P devices, each pack paying overhead T_o, with t per unit and a tail proportional to pack length:

$$T(b) \;\approx\; \frac{W\,T_o}{P\,b} \;+\; \frac{W\,t}{P} \;+\; \kappa\,b\,t. \tag{1}$$

Proposition 3.1 (Square-root law)**.** *Equation (1) is minimised at* $b^\star = \sqrt{W\,T_o/(P\,\kappa\,t)}$*, i.e. a pack time* $\tau^\star = \sqrt{T_o\,T_{ideal}/\kappa}$*, giving* $T^\star = T_{ideal} + 2\sqrt{\kappa\,T_o\,T_{ideal}}$ *with* $T_{ideal} = Wt/P$*.*

The optimum is flat ($2b^\star$ or $b^\star/2$ raise the overhead term by a quarter). A pull-based simulator (200 devices, log-normal speeds $\sigma = 0.5$, $W \in \{4{,}000, 40{,}000\}$, $T_o \in \{0.5, 2\}$, optional churn $1/600$ s^{-1}, 8 or 14 pack sizes, 10 seeds each) confirmed it: **(i)** with end-game hedging (Dean and Ghemawat, 2004) the fitted overhead term matches the prediction within 0.94–1.19×; **(ii)** ten times more work grows $\hat{b}^\star$ by 2.4–3.2× ($\sqrt{10} = 3.16$) and four times the overhead by 1.8–2.2× ($\sqrt{4} = 2$); **(iii)** hedging halves the tail coefficient (from 2.3–3.8 to 0.9–1.8) for 3–22% extra executions; **(iv)** one unit per pack is up to 2.7× slower than the best size; **(v)** speed-matched packs with hedging were fastest in all eight settings, 8–26% faster than hedging alone without churn but only 2–8% with churn. The dispatcher therefore sizes packs by *time*, per device,

$$\tau_d = \mathrm{clamp}\Big(\sqrt{T_o T_{\mathrm{ideal}}/\kappa},\;\; 9\,T_o,\;\; \min\left(0.05/\lambda_d,\; \mathrm{SLO}\right)\Big), \qquad b_d = \max\left(1, \lfloor \tau_d / t_d \rfloor\right), \tag{2}$$

where $9T_o$ keeps efficiency at least 90% and $0.05/\lambda_d$ keeps the chance that device d stays until the pack ends at least 95%. Packs land between about ten seconds and five minutes of compute.

3.3 Verification at a Few Percent

Replication is affordable for small CPU packs but not for GPU inference, which is to be checked by audit. Both audits below are specified and measured offline (Section 5); neither is in the program yet, which today repeats a share of whole prompts on a second computer instead (Table 1). **Diffusion trace audit:** the initial noise comes from the pack seed by a deterministic CPU generator; the worker commits the Merkle root of its latents, and an audit recomputes one random step (x_k, x_{k-1}) within a tolerance calibrated across GPU types. It costs one step in S and catches a worker that faked a fraction g of steps with probability g. **Teacher-forced LLM audit:** sampling is seeded, $y_i = \arg\max_v(z_{i,v}/T + G_{i,v})$ with Gumbel noise from the pack seed, so the validator checks an answer in one parallel prefill, tolerating differences only where the top two choices are within a calibrated margin.

Proposition 3.2 (Escrow rule)**.** *Hold each worker's last H pack credits R in escrow, forfeited on a failed audit. If audits occur at rate a and detect a fake with probability p_d, and skipping a pack saves at most $c \le R$, cheating is unprofitable whenever $a \ge 1/(p_d(1+H))$.*

Proof. A faked pack earns $(1 - ap_d)R - ap_dHR$ in expectation against $R - c$ for an honest one; cheating does not pay iff $ap_d(1+H)R \ge c$, which holds for all $c \le R$ when $ap_d(1+H) \ge 1$. □

With $H = 49$, an audit rate of 2% deters whole-output fakes and 4% deters fakes of half the steps. Weights are the largest fixed cost (an 8B LLM at 4-bit takes ≈6.5 minutes at 12.5 MB/s), so a device joins one lane per session and caches weights by hash.

4 QuickMode, BatchMode and SherpaHub

Every job declares *when* it must finish and *how* it can be cut. The **QuickMode** queue serves a waiting person in seconds; the **BatchMode** queue serves jobs that can wait hours or days (science, long videos). **WholeMode** sends a job uncut to one program (a WorkPack of one unit); **Split** cuts it into many WorkPacks. The choices are independent, so one pipeline (job → queue → units → WorkPacks → devices) serves all four cases; QuickMode is always served first and BatchMode fills every gap. Contributors are of three kinds: a browser tab, the regular program, and the same program started as a *SherpaHub*. A regular program *does* the work; a SherpaHub *organises* it.

Why BatchMode changes the communication limit. The argument of Section 3 is about waiting time. In BatchMode nobody waits: a SherpaHub keeps hundreds of units in flight, so while one waits on the network others compute, and only the *total* maths and bytes remain.

SherpaHub. A SherpaHub holds a model's weights and runs its flow step by step, but cuts each large matrix multiply into *maths packs* ("multiply this block by that block") that browsers compute in their WebAssembly sandbox with no model installed. Blocks use int8 inputs and exact 32-bit sums, so every device returns bit-identical results. A product $C = AB$ is checked with Freivalds' method (Freivalds, 1977): test $A(Br) = Cr$ for a random r in $O(n^2)$ instead of $O(n^3)$. With the entries of r drawn uniformly from a set S of integers and exact integer arithmetic, a wrong C passes one test with probability at most $1/|S|$ (Schwartz–Zippel); we use $S = [-2^{15}, 2^{15})$ and 64-bit sums, so 2^{-16} per round, and the SherpaHub checks *every* pack with no replicas. It keeps the small parts (normalisation, softmax, encoding) itself; attention, with no weights to cache, is the open problem. A SherpaHub needs memory, disk, uptime,

bandwidth and trust rather than GPU power, so it may be a CPU-only computer; running one is opt-in, with a data limit set by its owner.

What is built. Three of four stages are built and tested; the fourth, real model work cut into maths packs, is not. (1) **Kernels that agree everywhere**: the int8 multiply in the program (SSE2 or NEON), as a 655-byte WebAssembly SIMD module and in plain JavaScript is bit-identical on fixed test vectors in each of the five builds (`-workpack-check`); per core on one AMD EPYC server the program does 57.9, browser SIMD 24.7 and plain JavaScript 2.05 billion operations a second; two Freivalds rounds let a wrong result pass with probability at most $(2^{-16})^2 = 2^{-32}$. (2) **The SherpaHub**, a pure state machine that cuts a job, sizes packs, checks every result, strikes and bans liars, checkpoints and assembles; with up to 2,000 slow, leaving, lying or message-losing simulated devices it gave an exact result in every scenario, at **1.09 times the ideal time** with 0.1% of the work done twice when each device had ample work (about 2× with little work per device, and 1.0–2.7× under liars, departures and hub crashes). (3) **The website's broker**: the SherpaHub and every device keep one outbound WebSocket to a Cloudflare Durable Object that relays binary messages unread (1 MiB a message, 2,000 devices a SherpaHub). End to end, three browser-code devices and three programs computed a 4,096-unit job, liars were banned after three strikes, and a SherpaHub killed in a 16,384-unit job continued after a restart with the 4,476 units already done.

Bytes per operation: what limits Split. Hiding latency removes waiting, not bytes. A unit $C_{ij} = A_i B_j$ with $T \times T$ tiles and inner dimension K costs $2T^2K$ operations and returns $4T^2$ bytes of exact 32-bit sums, plus up to $2TK$ bytes of int8 inputs when neither block is cached. The results alone carry one byte per $K/2$ operations, so a device with upload $B_\uparrow$ bytes a second, and a SherpaHub with download $B_\downarrow$, can sustain at most

$$R_{\text{device}} \le \tfrac{K}{2}\, B_\uparrow, \qquad R_{\text{job}} \le \tfrac{K}{2}\, B_\downarrow \tag{3}$$

operations a second. With $K = 4{,}096$, a browser core at 25 billion operations a second would upload 12 MB/s, but a 2.5 MB/s home upload allows 5.1 billion per *device*, a fifth of one core however many cores it has; and each Gbit/s of the SherpaHub's link caps a job at 256 billion operations a second, about four of the program's EPYC cores. Section 3's conclusion therefore holds in BatchMode too: spreading a model's matrix multiplies over home connections (stage 4) is bandwidth-bound by Equation (3), which is why stage 4 is not built. Split maths packs suit jobs with a large inner dimension or a small output, and devices on fast links; exact int32 results cannot be shrunk without giving up the exact check.

Live split jobs (31 October 2026). Two matrix jobs ran on the operator's rented server, on four program copies of four cores each (no browser joined); Table 3 gives the results. The result ($4MN$ bytes, 16 GiB for the second job) is kept on the SherpaHub's disk and deleted once the final check passes, so a job's real size limit is that disk's free space.

Why the live jobs were slow. The second job reached 13.7 billion operations a second on 16 cores, about 1.5% of the 926 billion that the kernel's measured speed allows (its 42 min 44 s include about 30 s before the first device joined); one core alone would have needed about ten minutes instead of 43. The kernel is not the cause; the protocol is. A result must fit in one broker message (1 MiB), so a pack holds at most 15 units of 128×128 (10.8 on average), about 9 ms of work on four cores; each device has one pack in flight; and every message, including the 512 KiB input blocks, crosses the Cloudflare broker although the SherpaHub and the devices share one server. From the job's totals each device finished a pack every $4 \times 2{,}564/24{,}359 = 0.42$ s and spent about 6.3 ms of it computing, so it was busy 1.5% of the

time, which is the measured shortfall. These packs were capped by the 1 MiB message limit far below the $9T_o$ floor of Equation (2), whose efficiency bound $6.3/(6.3+415) \approx 1.5\%$ predicts exactly this. The breakdown is inferred from the counters and the code. The fix, several packs in flight per device and reuse of the input blocks a device holds, is now built and tested in the SherpaHub simulator (8.9× faster on the live job's shape); a live rerun is pending. Equation (3) sets the limit that remains, since keeping up with four cores would take about 113 MB/s of results per device. The live jobs show correctness (every unit checked, no wrong answer), not speed.

Table 3: Split matrix jobs on the live servers, 31 October 2026. Job 1's final pack count was not logged: 79 had been sent by 2,493 of its 4,096 units.

	Job 1	**Job 2**
Product ($M \times K \times N$), tile	4096^3, 64×64	$65{,}536 \times 4{,}096 \times 65{,}536$, 128×128
Units / operations	4,096 / 137 billion	262,144 / 35,184 billion
Packs	≥ 79	24,359 (33 re-sent after a timeout)
Time	about 15–20 s	42 min 44 s (13.7 billion operations/s)
Share of the kernel's speed (16 cores)	about 1%	1.5%
Wrong answers / strikes / final check	0 / 0 / passed	0 / 0 / passed

In-house SherpaHubs. The operator's own SherpaHubs also run batches of whole prompts (WholeMode), with 10% made a second time on a different computer with the same seed, and two switches: *SupplySimulation* (also contribute) and *DemandSimulation* (also send low-priority test jobs). In an eight-hour side-by-side run on 30 October 2026 against five dedicated load scripts on the same server (Table 18), its jobs were answered almost as often and as fast, by the same five computers (its longer tail is expected: they are low priority); the scripts were then retired. A 512×512 Z-Image Turbo picture took 1,037 s on about nine CPU cores of that server.

5 GPU Measurements

We ran the benchmarks on three RunPod GPUs that stand in for volunteer classes: an RTX 3090 (older gaming desktop), an RTX 4090 (current gaming desktop) and an A100 80GB (server), with the same script on each (PyTorch 2.4.1, CUDA 12.4, diffusers 0.32.2, transformers 4.46.3, vLLM 0.6.3); every GPU then audited the other two GPUs' outputs. Scripts and raw results are in the repository (`https://github.com/syncaissa/planetCompute.cpp`).

Throughput (B1). An SDXL image takes 3.7 s on the 4090 and 9.4 s on the 3090. A 7B model decodes 122–155 tokens per second for one request (6.5–8 ms per token, close to the 5 ms assumed in Table 2) and 2,200–3,200 at batch 32, which is why batch chat suits volunteers. MusicGen ran faster on the 3090 pod because its generation loop is bound by the host CPU.

Reproducibility (B4, B5). The same seeded SDXL job rerun on the same GPU gave a byte-identical image on all three GPUs. Across types no image was bit-identical, but all matched at SSIM 0.986–0.998 with a mean pixel difference of at most 1.1 of 255 (Table 5). Whole trajectories drift about 1% by the last step, but a *single* step recomputed from the worker's own latent agrees within 5.3×10^{-4} on every pair, so the trace audit uses a tolerance of 10^{-3}. Teacher-forced audits of seeded Qwen2.5-7B sampling (fp16, temperature 0.8, 16 answers of 256 tokens, 4,096 tokens per pair) reproduced 99.88–99.95% of tokens across GPU types and 99.90–99.95% on the same type (Table 7); every disagreement on every pair occurred where the

Table 4: Measured time per work unit (B1). LLM throughput uses vLLM with 4-bit AWQ models and 512 output tokens per request; 32B does not fit in 24 GB. LTX-Video on the 24 GB cards ran with CPU offload.

Measurement	RTX 3090	RTX 4090	A100 80GB
SDXL 1024^2, 30 steps: s/image (batch 1)	9.37	3.74	3.24
SDXL: s/image (batch 4)	9.04	3.64	2.92
SDXL: s/denoising step	0.312	0.125	0.108
LLM 7B AWQ: tokens/s, batch 1	122	146	155
LLM 7B AWQ: tokens/s, batch 32	2216	3183	3159
LLM 14B AWQ: tokens/s, batch 1	70	83	98
LLM 14B AWQ: tokens/s, batch 32	1228	1900	2100
LLM 32B AWQ: tokens/s, batch 1	–	–	55
LLM 32B AWQ: tokens/s, batch 32	–	–	1278
MusicGen small: s per 10 s audio (batch 1)	4.32	10.10	10.52
MusicGen medium: s per 10 s audio (batch 1)	8.19	20.97	19.55
MusicGen medium: s per 10 s audio (batch 4)	2.39	5.05	4.97
LTX-Video: s per 49-frame 704×480 chunk	22.0	18.6	6.1
Load SDXL to GPU (s)	1.37	2.23	2.46

Table 5: SDXL across GPU types (4 prompts, fixed seeds, CPU-generated initial noise). The last column is the largest relative error when one GPU recomputes a single denoising step of another GPU's trace.

GPU pair	Bit-identical	Min SSIM	Max mean $\lvert\Delta\rvert$ (0–255)	Max one-step audit error
RTX 3090 vs RTX 4090	0/4	0.9876	0.96	0.00052
RTX 3090 vs A100 80GB	0/4	0.9890	0.95	0.00052
RTX 4090 vs A100 80GB	0/4	0.9860	1.11	0.00053

two best choices were within 0.035. The audit therefore excuses only positions with margin below $\delta = 0.05$; fewer than 2% of positions have a margin below 0.1.

Table 6: Cost of one audit (B7). Diffusion: recomputing one denoising step, as a fraction of generating the image. LLM: one teacher-forced pass (fp16) over a 256-token answer, as a fraction of vLLM decoding the same number of tokens at batch 1 (4-bit AWQ, Table 4); an fp16 decode would make the fraction smaller.

GPU	Diffusion audit (s)	of one image	LLM audit (s)	of decoding the answer
RTX 3090	0.313	3.3%	0.164	7.9%
RTX 4090	0.158	4.2%	0.154	8.8%
A100 80GB	0.143	4.4%	0.148	8.9%

Audit cost (B7). Measured offline in PyTorch and vLLM, auditing one diffusion step costs 3.3–4.4% of making the image, and auditing a whole LLM answer 7.9–8.9% of decoding it (Table 6). At the 2–4% audit rates of Proposition 3.2, verification would add about 0.1–0.4% to the grid's cost, against 400% for five-fold replication; this holds only once the audits run in llama.cpp and stable-diffusion.cpp with tolerances re-measured there.

What carries over to volunteer machines. The RTX 3090 and 4090 are the same GeForce consumer cards volunteers own. Table 7 states, per measurement, what transfers. Volunteers will run llama.cpp, stable-diffusion.cpp or WebGPU rather than PyTorch, so each lane's tolerances

are re-measured in its own runtime before it opens, and uncalibrated device classes run with full replication.

Table 7: Top: teacher-forced audits of seeded LLM sampling (Qwen2.5-7B-Instruct, fp16, temperature 0.8, 16 answers of 256 tokens, 4,096 tokens per pair): tokens that disagree, and in brackets the largest margin between the two best seeded scores at any disagreement. Bottom: how each GPU measurement carries over to volunteer laptops, desktops and servers.

Generated on ↓ audited on →	**RTX 3090**	**RTX 4090**	**A100 80GB**
RTX 3090	0.10% (0.035)	0.05% (0.010)	0.07% (0.021)
RTX 4090	0.07% (0.029)	0.05% (0.005)	0.05% (0.005)
A100 80GB	0.12% (0.035)	0.12% (0.030)	0.10% (0.030)

Measurement	**Carries over?**	**Why, and what is still needed**
Same-GPU bit-identical results (B4)	Directly	Decided by architecture, kernels and versions; same for a home card with the same stack
Cross-GPU tolerances (B4, B5)	Directly, for these architectures	Re-measure for AMD, Intel, Apple and other NVIDIA generations, and in each lane's runtime
Audit cost as a fraction of work (B7)	Directly	Audit and work slow down together on a slower machine
Seconds per image, token, chunk (B1)	As a scale	Laptops, politeness caps and heat lower it; each device is measured on joining
Speed spread between devices	Supports the model	Measured 2.5–2.9× SDXL gap, consistent with a wide spread such as the simulated $\sigma = 0.5$
Download, network, churn, electricity, owner's use	No	Data-centre links; not measured; the protocol does not depend on these numbers

6 Validation and Credit

PlanetCompute credits only *validated* work and *verified* capacity and combines mechanisms from the sabotage-tolerance literature (Sarmenta, 2002; Anderson, 2020).

Quorum and the collusion race. A unit is issued to distinct random devices until some result hash has q matching votes. If a fraction f of devices always returns the same wrong hash, acceptance is a race to q votes.

Proposition 6.1 (Collusion race)**.** *Under uniform random assignment and negligible honest error, the colluders' hash is accepted with probability*

$$P_{wrong}(f, q) = \sum_{k=0}^{q-1} \binom{q-1+k}{k} f^q (1-f)^k. \tag{4}$$

Proof. The colluders win iff their q-th vote arrives while honest votes number $k \le q-1$, which by the negative binomial distribution has probability $\binom{q-1+k}{k} f^q (1-f)^k$; sum over k. □

For $q = 2$ this is $f^2(3-2f)$, not the often quoted f^2: at $f = 0.2$, 10.4% rather than 4%. Quorum must therefore be paired with consequences. **Spot checks** insert known-answer units at rate s; a device cheating on a fraction c of units survives n units with probability $(1-sc)^n$. **Strikes:** a failed check or an outvoted result adds a strike; three strikes ban the device. **Retroactive invalidation:** on a ban, every unit the device helped accept is invalidated and re-queued.

Adaptive replication (Anderson, 2020): a device with $\geq T$ consecutive validated results may be accepted alone, except for a random audit fraction a (Algorithm 1).

Algorithm 1 Validator: quorum with strikes and retroactive invalidation (abbreviated)

Require: unit u, quorum q, trust threshold T, audit rate a, max replicas $R_{\max}$
$d_1 \leftarrow$ random eligible device
if streak$(d_1) \geq T$ **and** rand$() \geq a$ **then** accept $r(d_1)$; **return**
end if
repeat issue u to a new distinct device; on result r: $V[r] \leftarrow V[r] + 1$
until $\max_r V[r] \geq q$ **or** $R_{\max}$ replicas used
accept $r^\star \leftarrow \arg\max_r V[r]$
for all device d with result r_d **do**
    if $r_d = r^\star$ **then** streak(d) += 1; add u to contrib(d)
    else streak$(d) \leftarrow 0$; strikes(d) += 1
        if strikes$(d) \geq 3$ **then** ban d; re-queue all of contrib(d) ▷ retroactive
        end if
    end if
end for

Simulation. We simulated 2,000 devices and 10^5 units with honest error $e = 10^{-3}$, adversary fractions f from 0.01 to 0.20 and cheat probability $c \in \{1, 0.1\}$, with $T = 10$, $a = 0.10$ and $s = 0.05$. Table 8 gives the $f \in \{0.05, 0.20\}$ grid; the simulator and every run are in the repository. Without validation the wrong rate is about $f \cdot c + e$. With quorum, strikes and retroactive invalidation every colluder was banned and *no* wrong result remained among 10^5 units (an upper bound of 3×10^{-5} at 95% confidence); the penalties, not the larger quorum, do the work. Adaptive replication is cheaper at a small residual error. Honest devices were banned only because being outvoted by colluders also earns a strike; rescinding strikes caused by devices later banned removes this.

Table 8: Validation policies under collusion (10^5 units, 2,000 devices, seed 1): wrong results accepted (after retroactive invalidation) / executions per accepted unit (every execution, including spot checks and re-runs). Quorum banned every cheater in every cell; adaptive replication banned 100% of always-cheaters and 67% and 66% of stealthy ones. Honest devices banned: at most 6 under quorum, none under adaptive.

	Always cheat ($c = 1$)		**Stealthy** ($c = 0.1$)	
Policy	$f = 0.05$	$f = 0.20$	$f = 0.05$	$f = 0.20$
No validation	5.08% / 1.00	20.14% / 1.00	0.61% / 1.00	2.09% / 1.00
Quorum q=2	0 / 2.01	0 / 2.03	0 / 2.06	0 / 2.22
Quorum q=3	0 / 3.01	0 / 3.03	0 / 3.09	0 / 3.30
Adaptive (q=2, spot 5%)	0.10% / 1.25	0.08% / 1.25	0.22% / 1.28	0.63% / 1.36

Credit and ledger. Credit (shown to users as *Compute Credit*, in Work Units, WU) measures validated computation. Device d's credit is $C_d = \sum_{u \in \mathcal{V}_d} w_u$ over its validated units, where w_u is the unit's benchmark-normalised cost; a verified device that delivers no pack is credited from its verified rating (below), never from a self-reported speed. Credits are an append-only ledger in Amazon DynamoDB, keyed by `pack_id#device_id` so a retried report is never credited twice, with every credit of a pack and the running totals written in one transaction; clawbacks are reversing entries. A separate lifetime total, the sum of all credit ever earned, never decreases

when credit is spent. These credits are the PlanetComp points of the Executive Summary: the DynamoDB ledger is the working record of every pack, and credit can be transferred out as the PlanetComp token on the Solana blockchain at a published rate; users need no wallet unless they choose to transfer, and the design anchors the Merkle root of all balances on chain each epoch so that anyone can check their own balance. Credit is redeemed one for one as free processing time on the grid, $G_d = C_d - \sum_{j \in \mathcal{J}_d} w_j \geq 0$. Every check before a transfer, and the balance itself, live on the server; a program never computes its own credit and never holds a key.

Capacity check. Credit where no pack is delivered uses a *verified* rating, never a self-reported one. One Durable Object per device sends a capacity test about once a minute: k small chunks of the device's own kernel with random seeds, sized for about a second. It times the answer on its own clock, so network delay can only lower the measurement, and recomputes two random chunks; faking a fraction g of the chunks is caught with probability about $1 - (1 - g)^2$ per test, and three failed tests in a row make the device unverified. A test costs about 1.7% of the time.

Commit, then random audit. The capacity check shows what a machine *can* do, not that its reported packs were computed. The program and the browser console therefore earn through *leases* checked after the fact. A verified device receives a random base and a number of CPU WorkPacks of 4×10^6 samples each, sized for about ten minutes; every pack's seed follows from the base, so anyone can recompute any pack. When time is up the device *commits* the Merkle root of its per-pack results; only then does the website choose $K = 20$ packs at random, and the device must answer within a minute with each result and its Merkle path, which the website recomputes. A device that computed a fraction h of its packs honestly passes with probability h^K: 12% at $h = 0.9$, 10^{-6} at $h = 0.5$. A pass credits every pack; a fail credits nothing and is a strike, and three strikes in a row hold the device for an hour. GPU leases commit a Merkle *sum* tree over dispatches of up to 8,192 groups and are audited on 16 groups (plus two more of each picked dispatch); they are tested against the CPU code but have not yet run on real WebGPU hardware. For WorkPacks, every build that carries an identity is credited by these audits only (Section 10.1); grid chat answers (Section 10) are credited per token, with a quarter of them double-checked.

7 Competitive Dispatch

Video and other latency-sensitive jobs need the *last* pack fast. PlanetCompute can therefore send every WorkPack to n machines at once, in the spirit of speculative backup tasks (Dean and Ghemawat, 2004), and turn the redundancy into a competition (Algorithm 2): (1) each pack goes to n devices (default $n = 5$) on n distinct networks (IPv4 /24 or IPv6 /48 prefixes, where possible distinct autonomous systems) and accounts; (2) a finisher sends the Merkle root ρ_d of its result (plus a perceptual hash for GPU frames) on its own authenticated socket, which timestamps the finish, and answers a challenge for one random leaf of that tree; no result or root is ever shown to a volunteer; (3) a result is accepted once $q = 3$ commitments from three networks match; (4) the first finisher in the accepted cluster receives αR (default $\alpha = \frac{1}{2}$) and every other matching finisher shares $(1 - \alpha)R$ as consolation; (5) a non-matching result earns a strike, and three strikes ban the device and claw back what it helped accept. One Durable Object per project coordinates its packs, so its message order is the authoritative finishing order, with no distributed locks or clock comparisons.

Lazy copiers. Because the hybrid rule keeps crediting matching answers after acceptance, it invites a *lazy copier*: a device that learns the accepted hash, from a colluder or from the

consumer's side, and submits it without computing, to collect consolation credit (the lazy-participant problem of Golle and Mironov (2001)). Three rules close this: the dispatcher never shows a result, hash or root to any volunteer; credit follows the order in which roots arrive on each device's own socket, so a root cannot be replayed for another device; and the random-leaf challenge, answered within seconds, shows the device holds the whole result, not only its hash. The challenge is one extra round trip, so acceptance still happens at the q-th match. A copier must therefore receive the entire result from a colluder, which is the "one computation, many submissions" Sybil case of Table 14, limited by network-distinct dispatch and account binding. Because a pack's total credit is fixed at R, copying never creates credit; it only takes consolation from honest finishers. The simulations below did not model copiers.

Proposition 7.1 (Speed-race exposure)**.** *A fabricated answer costs nothing and arrives first. Without penalties, under first-q acceptance with n simultaneous replicas,* $P_{wrong} = \Pr[\mathrm{Bin}(n, f) \geq q] = \sum_{j=q}^{n} \binom{n}{j} f^j (1-f)^{n-j}$.

At $n = 5$, $q = 3$, $f = 0.2$ this is 5.8%; the simulation measures 6.1%. For $n = 2q - 1$ it equals Equation (4): with $2q - 1$ replicas one of the two answers must reach q, so the race is the same event.

Algorithm 2 Competitive WorkPack dispatch with hybrid acceptance (abbreviated)

Require: WorkPack u, replicas n, quorum q, credit R, winner share α, deadline t_{dl}

send u at once to n devices on pairwise distinct networks and accounts
while now $< t_{\mathrm{dl}}$ **and** some device has not reported **do**
    receive Merkle root (d, ρ); challenge one random leaf; if the answer fails, STRIKE d; else add d to $V[\rho]$ ▷ or SSIM for GPU frames
    if nothing accepted **and** $|V[\rho]| \geq q$ **then** accept $h^\star \leftarrow \rho$; deliver to the consumer only ▷ low latency
    end if
end while
credit the earliest finisher in $V[h^\star]$ with αR, the others in $V[h^\star]$ $(1-\alpha)R$ shared
STRIKE every device that reported $\rho \neq h^\star$ ▷ as in Algorithm 1

Proposition 7.2 (Network diversity)**.** *If the adversary's devices lie on S networks and each pack uses n distinct networks, a pack holds at most* $\min(n, S)$ *adversarial replicas; hence if* $S < q$, $P_{wrong} = 0$.

This turns a cheap attack (many virtual machines in one data centre) into an expensive one; since residential proxies sell network diversity, the penalties remain.

Matching AI outputs. CPU packs match by SHA-256. GPU inference is not bit-reproducible across vendors, so GPU outputs match when perceptually identical, by SSIM (Wang et al., 2004) against a threshold set per model. The same request run by the program on Linux with an RTX 4090 and on Windows with a Tesla T4 (Table 9) shows why both points matter: exact matching would reject honest work, and a single threshold is wrong, since 0.98 accepts both Stable Diffusion pairs but rejects both honest FLUX pairs, while a wrong picture scores near 0.3. Provisional thresholds are about 0.90 for FLUX.1 schnell and 0.97 for Stable Diffusion 1.5, carried in each model edition's metadata.

Simulation. We simulated 2,000 devices with log-normal speeds (median compute time 1.0), 5% honest drop-out and 10^{-3} honest error; colluders ($f = 0.2$, spread over 400 networks, the hardest case) answer instantly with a common fake. Each configuration processed 20,000

Table 9: The same request on two machines (Linux + RTX 4090 vs Windows + Tesla T4): how alike the pictures are. Pixel differences on the 0–255 scale; SSIM on grey levels, 7×7 window.

Picture (model, seed)	Pixels equal	Mean difference	Within 8 levels	SSIM
FLUX.1 schnell, cat (13)	4.3 %	4.07	81.8 %	0.941
FLUX.1 schnell, sign (14)	1.4 %	4.43	79.2 %	0.956
Stable Diffusion 1.5, lighthouse (1001)	9.6 %	1.17	98.3 %	0.987
Stable Diffusion 1.5, apple (1002)	5.8 %	1.61	97.3 %	0.992
Different pictures, same model (for contrast)				0.24–0.35

WorkPacks (Table 10). **(i)** The naive rule is exploitable as Proposition 7.1 predicts (6.09% wrong). **(ii)** Three-of-three unanimity silently re-issues mixed packs, so 1.80% are wrong and no colluder is ever out-voted; spare replicas ($n > q$) are what expose cheaters. **(iii)** With $n = 5$ and penalties every colluder was banned and no wrong pack survived, for all three acceptance rules. **(iv)** First-three-matching is fast (1.10) but credits the fastest quartile with 9.6× what the slowest receives; waiting for all is fair (2.7×) but slow (1.90); the **hybrid** rule, which accepts at the third match but keeps crediting later matching answers, is both (1.10, 2.7×). **(v)** With the adversary on two networks, wrong acceptance was zero in every configuration. The cost is about 5 executions per pack against 3.0–3.3 for sequential quorum with the same $q = 3$ (Table 8), so racing is for latency-sensitive jobs.

Table 10: Competitive dispatch, $f = 0.20$, $q = 3$, adversary on 400 networks. "Latency" is time to acceptance in units of the median device's compute time. "Fast/slow credit" is the mean credit of the fastest quartile of honest devices divided by that of the slowest quartile.

Dispatch	Acceptance	Penalties	Wrong accepted	Exec./pack	Latency	Fast/slow credit
$n = 3$	Unanimous (3 of 3)	✗	1.80%	6.67	1.64	1.7×
$n = 3$	Unanimous (3 of 3)	✓	1.80%	6.67	1.64	1.7×
$n = 5$	First 3 matching	✗	6.09%	5.19	1.22	4.2×
$n = 5$	First 3 matching	✓	0	5.03	1.10	9.6×
$n = 5$	Wait for all	✗	6.09%	5.19	1.79	2.3×
$n = 5$	Wait for all	✓	0	5.03	1.90	2.7×
$n = 5$	Hybrid (ours)	✗	6.09%	5.19	1.22	2.3×
$n = 5$	Hybrid (ours)	✓	0	5.03	1.10	2.7×

8 Orchestration at a Million Devices

The orchestrator connects consumers' jobs to volunteers' devices through Cloudflare Durable Objects (Cloudflare, 2026): single-threaded, strongly consistent objects with their own storage. It is designed and simulated, not yet deployed. Large data never passes through an object; inputs, outputs and weights move by presigned URLs. The simulation refined it into five kinds of object: **lane shards**, each holding the WebSockets of about 10^3 warm devices of one lane and running their capacity checks; sharded **lane directories**, each serving about 25 shards and knowing which jobs have packs waiting; a **planner** that moves devices between lanes every 15 s; a stateless **intake** Worker; and one **coordinator per job**.

Method. The objects' logic reads no clock and touches no network or storage, so a seeded discrete-event simulator (Node.js) runs the very classes that will be deployed, charging 0.5 ms per message plus 0.02 ms per batched item, 2–10 ms between objects and about 40 ms to a device. Devices are 70% browser CPU, 25% browser WebGPU and 5% desktop GPUs (12 or 24 GB), with log-normal speeds and sessions (median 40 minutes); about three share a

network address; 2% cheat (70% of their packs wrong) and 0.2% never answer. Five lanes (CPU, embeddings, an 8B LLM, SDXL, a 14B LLM) receive log-normal jobs (median 2,000 device-seconds) at half of capacity. 80% of honest devices start with an earned reputation and cheaters start untrusted; new devices are never trusted alone; trusted ones are audited at 5% or more; late packs are hedged; a caught cheater is banned and its recent packs re-run. *Push* spreads packs over shards at start, *steal* lets empty shards take half the busiest queue, and *pull* lets shards with idle devices ask for batches.

Table 11: Orchestrator simulation, one simulated hour each. "Idle while queued" is device time spent idle while packs for the same lane waited, in the worst lane. Times are time to finish, at the 99th percentile, for small ($< 10^3$ device-seconds) and large ($\geq 3 \times 10^4$) jobs. All runs use pull unless named otherwise, with 10^4 consumers, and precede the fixes described below; the million-device run with the fixes is described in the text.

Scenario	**Devices online**	**Idle while queued**	**Stalled jobs**	**Small p99 (s)**	**Large p99 (s)**
Pull, 10^6 volunteers (first run, before the fixes)	916,831	0.26%	0	144	417
Pull, 10^5 volunteers (baseline)	91,563	0.22%	1	149	513
Push	91,646	**32.9%**	**4,466**	857	1,660
Steal	91,667	1.47%	0	151	534
Deploy at 30 min, 30-s reconnect jitter	90,973	0.51%	0	150	488
30% of devices lost at once	88,084	0.55%	0	151	464
Burst: 2,000 jobs of 5×10^4 device-s	91,261	0.41%	23	154	684
Only 40 network addresses	91,632	0.32%	0	147	496
Demand at 90% of capacity	91,855	0.01%	**6,319**	1,869	2,623

Results and faults found. Push grid-locked: a third of device time was wasted because packs sat in shards whose devices had left; stealing repaired most of this and pull removed it (Table 11). The simulator found seven faults before any deployment, among them a single directory per lane that was 67% busy at 10^5 devices (now sharded), 95% of pull requests coming back empty, a livelock of about 2,500 requests a second, and about 1.2×10^4 multi-gigabyte model downloads an hour (moves now prefer devices that hold the model). Three later fixes: *sleeper cheaters*, which earn trust and then cheat, had 5,278 wrong answers accepted in an hour at 10^5 devices; with audits that fall with a device's record but never below 5%, and re-running a caught device's last 256 packs, accepted wrong answers fell to 2,031, of which 1,947 were corrected after delivery and 84 never were (all from sleepers not yet caught when the hour ended); a further 2,049 were fixed before delivery. Admission now refuses a job whose estimated wait exceeds five minutes (burst stalls fell from 23 to 3, while refused submissions rose from 2,815 to 7,975), and spare copies are cancelled on acceptance. Still open: under 90% demand, 4,633 admitted jobs still stalled (from 6,319).

Faults in the platform. With 1% of requests between objects refused, 0.2% of WebSocket messages lost, 0.5 crashes per object per hour and 0.2% of replies lost after delivery, three designs were compared at 20,000 devices (Table 12). The simple design needs no queue service and no load balancer, only four rules: keep a message until acknowledged and resend with growing pauses; give every message an ID and drop repeats; save state before acknowledging; re-issue a long-overdue copy.

One million devices, and the cost. In one simulated hour with the fixes above, at about a million devices (1,151,633 came and went, 917,629 online on average) and 10^4 consumers, 253,881 of 261,125 jobs finished within the hour and the rest were still running; none was refused

Table 12: The orchestrator under faults (20,000 devices, one hour).

Design	Jobs finished	Jobs lost	Credit wrong	Processed twice
Nothing retried	4,360 of 6,573	493	10,202 units missing	0
Retries without message IDs	6,167 of 6,584	113	1,123 units paid twice	6,095
Simple design (four rules)	6,426 of 6,637	0	0 of 1,051,195	0

or stalled; 186,057 cheating answers were caught and none was delivered uncorrected; and the busiest of 42 lane directories peaked at 411 messages a second. This run had the 2% ordinary cheaters of the Method; sleeper cheaters were tested only at 10^5 devices (above), where 84 wrong answers remained. It is a single run with one seed. The cost runs of Table 13 were made separately, with the same seed and cost accounting added, so their job counts differ slightly from this run's. Priced at list rates around the clock, this original design costs about $111.54 an hour with volunteers' status reports sent on the open WebSocket every five minutes ($195.70 with reports to the website each minute, as first built), because about 8,700 objects stay awake, objects exchange about four messages per WorkPack, and the ledger takes a write per credited device per minute (Table 13). Against this, with the same five-minute reports ($111.54 is the run re-priced with them), a **low-cost design** changes five things, each a simulator switch: the ledger rolled up hourly, job groups instead of one object per job, shards of 10^4 devices, batching of messages between objects within 250 ms, and multi-pack leases for trusted devices. Its first million-device run delivered 4 wrong results uncorrected, because cheaters holding leases filled packs up to the copy limit, which were then accepted without enough matching answers; never accepting a pack without its matching answers, and leasing to trusted devices only (the first run also leased to untrusted ones), fixed it. The fixed design costs about $9.80 an hour, 11.4 times less than the original with the same status reports; the price is latency (small jobs finish in a median of 226 s instead of 74 s) and idle capacity: in the CPU lane devices sat idle while packs waited 33% of the time (the original design: under 0.3%), because leases and batching hold work in a few places, as push did. It suits batch work, and the idle lane is the next fault to fix.

Table 13: One simulated hour at about a million volunteer devices and 10^4 consumers (demand at half of capacity), at list prices. Same seed in all three runs, which are separate from the run described in the text.

	Original design	Low-cost, first run	Low-cost, with the fix
Cost/hour, reports on the WebSocket every 5 min	$111.54	$9.82	$9.80
Objects awake on average	8,743	213	214
Requests between objects per second (billed)	49,458	11,135	11,483
Device messages per second	20,432	5,984	6,962
Ledger writes per second	14,686	683	579
Jobs submitted / finished within the hour	261,385 / 254,027	261,491 / 230,708	261,681 / 236,650
Jobs refused / stalled (> 600 s)	0 / 0	0 / 281	0 / 76
Small jobs: time to finish, median / 99th pct.	74 / 152 s	249 / 767 s	226 / 599 s
First result to the consumer, median / 99th pct.	7 / 21 s	58 / 262 s	51 / 210 s
Extra copies (quorum, audits, hedges)	22.5%	40.3%	30.7%
Accepted without enough matching answers	0	680	0
Wrong answers delivered and never corrected	0	4	0

What it does not show, and what is next. The simulation does not measure Cloudflare's real latency, real model speeds or real browsers and programs; device behaviour and cheating are modelled. The steps from simulation to a real test are in Table 21 (E9). Clients must reconnect with random delay after an update: without it, 10^5 simulated devices reconnected at

about 9×10^4 connections a second, against about 3×10^3 with 30 s of jitter. The design is not tied to Cloudflare: its logic is written against five small interfaces (actor runtime, object store, database, mailer, secrets), with open-source equivalents on plain Ubuntu servers, which then need their own load balancers and a durable message queue.

9 Threat Model and Privacy

Table 14 lists the main threats. The Coinhive lesson (Eskandari et al., 2018; Konoth et al., 2018) is that background compute was condemned for being *covert*; the dispatcher refuses devices without recorded consent, and both hosts always show what runs and a pause control.

Table 14: Threats and mitigations.

Threat	**Mitigation**
Malicious job or hostile model file	No-import WASM in the browser; WebGPU shader validation; desktop engines in their own processes in the OS sandbox (models read-only, no network, no other programs, no sign-in)
Tampered module or update	Every build signed with Ed25519 and checked before install (third-party modules checked by SHA-256 against a signed manifest: specified); every build signed with a key kept off the website, storage and build service; servers install only signed builds; installers to be code-signed
Stolen sign-in	No passwords (one-time codes by email); email on a sign-in from a new browser or country; list of sign-ins with sign-out; a computer's link bound to its device key
Leaked keys; exposed prompts	Keys held as encrypted secrets of the website, with rotation; no prompt written into shared files
Free file storage	Only files the account's program made (SHA-512 registry), only PNG/WebM at the program's sizes, daily limits per account and network
Lazy copier (submits another's answer)	Results never shown to volunteers, roots bound to each device's socket, a random Merkle-leaf challenge, fixed credit per pack (Section 7)
Cheating volunteer, fast fakes	Built: commit-then-random-audit, Freivalds checks, repeated prompts, strikes and bans. Simulated: quorum, spare replicas $n > q$, network-distinct quorum, retroactive invalidation and clawback (Sections 6, 7)
Sybil devices; one computation, many submissions	Account-bound device keys, rate limits on new accounts; per-account replica exclusion (specified) (a group that computes once returns a correct result but defeats independence)
Old or modified program keeps earning	Credit and transfer checks only on the server; builds with an identity credited by audits only; a minimum build for earning (Section 10.1)
Data exposure to volunteers	Privacy tiers (Table 15)
Perceived cryptojacking; antivirus	Explicit opt-in, visible indicator, one-click pause, open-source client, no silent auto-start, conservative default caps; code signing and vendor allow-list submissions planned

What volunteers can see. Encryption in transit is easy; computing on encrypted data at speed is not. Fully homomorphic encryption remains orders of magnitude slower for neural networks, multi-party computation needs round trips per layer, and practical attested GPU enclaves exist only on server hardware. PlanetCompute therefore offers *rotation obfuscation*: for secret orthogonal Q, R, $x' = Rx$ and $W' = QWR^\top$ give $W'x' = Q(Wx)$, and in transformers the rotation can be folded into all weights in advance at no run-time cost (Ashkboos et al., 2024a,b). A longer key does not help against a public model: orthogonal Procrustes recovers R from the published and rotated embeddings in one SVD. What helps is private weights, decoders kept on the consumer's side, and limiting the blast radius: G key groups, keys changed every

period T_k, and fragments only, so one broken key exposes fragments of about 1% of one lane for one day with $G = 100$. Every job declares a tier (Table 15); PlanetCompute does not claim that volunteers cannot decode Protected jobs.

Table 15: Privacy tiers.

Tier	Computed by	What a volunteer can see	Cost
Public	Any browser or app	The data itself; only non-confidential jobs are eligible	Lowest
Protected	Desktop app only	Rotated numbers in fragments, in memory only, encrypted in transit; a determined expert with the public model could decode one key's share. Not encryption during processing.	Low
Confidential	Attested server TEEs, or FHE for small models	Nothing, within the guarantees of the attested hardware	Higher

9.1 Consent, Cost and Safety for Volunteers

What the volunteer sees. Nothing runs until the volunteer ticks a consent box, which names the shared cores, caps, status reports and that electricity and wear are the volunteer's, and presses *Start computing*; a new session needs new consent. Whenever the console is closed, a status light (Running, Idle, Paused, Stopped) with PlanetCompute's own CPU, GPU and memory use and the credit received stays in the page header; *Pause* keeps the session, and exit asks for confirmation. The program shows the same console in a window and keeps a tray icon. Table 16 shows what each host can observe and uses today.

Table 16: Observability of the eligibility predicate. ✓ yes, ∼ partly, ✗ no.

Signal	Browser can observe	Desktop can observe	Used today
Consent	✓ (in-page)	✓	used
Cores, CPU cap	✓	✓	used
User idle	∼ Idle Detection API (Chromium)	✓ OS idle time	specified
AC power	∼ Battery Status API (Chromium)	✓ OS power source	specified
Allowed hours	✓	✓	specified
Thermal limit	✗	∼ where sensors exist	specified
Other-process load	∼ Compute Pressure (Chromium)	✓ per-process CPU	specified
Runs when closed	✗	✓ tray	used

What it costs the volunteer. The marginal cost is electricity, $\Delta P \cdot h$ for extra draw ΔP over h hours (Table 17; assumed values, to be measured, E6 in Table 21). Electricity and wear are the volunteer's: the terms, the consent text, the program's agreement box and `-help` all say that sharing compute uses electricity and may add to the wear of fans, batteries, processors and graphics cards, that these costs are not reimbursed, and that PlanetCompute.ai is not liable for them. PlanetCompute is not a source of income; a volunteer's electricity buys credit redeemed as processing time on the grid. Every model licence in the library allows redistribution from PlanetCompute's own storage, with conditions (Table 17); non-commercial models are not added.

Table 17: Left: illustrative extra electricity in one 8-hour night. Right: licences of the library's models (a reading of the licences, to be confirmed by counsel before launch).

Device	ΔP	Energy
Laptop CPU, all cores	25 W	0.20 kWh
Desktop CPU, all cores	80 W	0.64 kWh
Desktop discrete GPU	200 W	1.60 kWh

Licence	Library models; conditions
Apache-2.0	Qwen, FLUX.1 schnell, Z-Image Turbo, Wan 2.1, T5/umt5; ship licence, mark changes
OpenRAIL-M	Stable Diffusion 1.5; pass on its use restrictions
MIT	CLIP-L, DeepSeek-R1 distillation; keep notice
Llama 3.3 Community	DeepSeek-R1-Distill-Llama-70B; "Built with Llama", Meta's use policy
Non-commercial	none (e.g. FLUX.1 dev): not added

10 One Program for Both Sides: `planetCompute`

`planetCompute` is one Rust and Tauri 2 (Tauri Programme within The Commons Conservancy, 2024) program with llama.cpp (Gerganov and llama.cpp contributors, 2023) linked in. It **contributes** when idle (on a desktop it shows the same volunteer console as the website; on a server, `-headless` runs a native engine that computes the same CPU WorkPacks, checked against shared test vectors on every build) and **harnesses** when its owner needs answers: a local OpenAI-style API (`/v1/chat/completions`, `/v1/images/generations`) for chat with llama.cpp and for pictures and video with stable-diffusion.cpp (leejet and contributors, 2026), run as a separate process. The library holds open-weight models, each file checked by SHA-256 against its publisher's, so every device that runs one model ID runs the same bytes and answers can be compared. Jobs run on this computer, or in the PlanetCompute pool, supplied by contributors' programs and, for now, by the operator's in-house SherpaHubs.

Grid, step one. A chat job sent to `POST /v1/jobs` goes to an idle volunteer program that holds the model; it runs at temperature 0 with a seed chosen by the website, and with probability 0.25 a second program runs it too and the answers must match exactly. A live load test with five scripts and two program copies first saw answers of 15–110 s; the cause was llama.cpp using every hardware thread on a loaded server, and limiting grid jobs to the shared cores fixed it (Table 18); the first double-checked job agreed.

A sandboxed native engine. A job carries a model ID and inputs, never code; but a model file from outside could exploit a parser bug. The program therefore never opens a model itself: a separate *model engine* and the image engine run in the operating system's sandbox (Landlock and seccomp on Linux, a sandbox profile on macOS, an AppContainer in a one-process job on Windows), may read only the models and their own folder, write only the pictures folder (the model engine writes nothing), and have no network, no other programs and no sign-in. `planetCompute -sandbox-check` makes each engine try to read outside its folders, list the home folder, write outside, connect to the internet and start a shell; every attempt must fail, and CI runs it on all five builds.

First measurements. Table 18 collects chat on a rented RTX A4000, pictures and video, the grid-job fix and the eight-hour run of an in-house SherpaHub against the load scripts; on the A4000 the first token arrived after 0.10 s and library downloads ran at 17–18 MB/s.

Pictures and video ran on an RTX 4090 under Linux and on a Tesla T4 under Windows (`planetCompute.exe`). All ten RTX 4090 jobs ran on the GPU at 68–100% use with no CPU fallback; the T4 took 2.9–4.2 times as long for pictures and 6 times as long for the clip; on a 4-core CPU the CPU build made a 256×256, 4-step picture in 62.9 s (Table 18). On the virtual GPU of an Apple M1 CI runner Metal fails and the CPU made the picture (46 s).

Table 18: Measured on real machines: the same program on rented servers and the operator's live server. Pictures and video through the Harness API (Vulkan builds); the first job of each model includes loading it, "then" is every later job; GPU memory as used on the RTX 4090. The CPU build on the T4 machine's 4 cores made a 256×256, 4-step Stable Diffusion 1.5 picture in 62.9 s.

Chat, RTX A4000 (16 GB), 4-bit, all layers on the GPU in the Vulkan build: tokens/s		
	Vulkan build	**CPU build**
Qwen3 4B Instruct	83.1	23.6 (3.5×)
Qwen2.5-Coder 7B	58.9	17.4 (3.4×)
Pictures and video: seconds		
	RTX 4090, Linux	**Tesla T4, Windows**
FLUX.1 schnell 4-bit, 1024×1024 / 1024×768, 4 steps (12–14 GB)	38.7, then 9.4–12.0	111.4, then 36.8
Stable Diffusion 1.5 8-bit, 512×512, 20 steps (4 GB)	21.2, then 4.4–4.5	89.8, then 15.9
Wan 2.1 T2V 1.3B 8-bit, 2 s clip, 832×480, 33 frames, 20 steps (10 GB)	87.8 and 77.2	492.5
Grid jobs, five scripts to two program copies (four shared cores each, Qwen2.5 0.5B, 4-bit)		
	Every core	**Shared cores**
Run time of an answer, median (slowest)	36.8 (109.5) s	0.8 (1.9) s
Wait in the queue / total time per job, medians	49.4 / 110.2 s	0.03 / 2.2 s
Server load (16 threads)	about 30	about 9
Eight hours side by side, 30 October 2026, 07:45–15:42 UTC (chat jobs, qwen2.5-0.5b)		
	5 load scripts	**1 SherpaHub**
Chat jobs sent (an hour)	5,769 (730)	4,384 (555; set 600)
Answered	99.9%	99.8%
Time to answer, median / 95th percentile	3.2 / 25 s	3.5 / 41 s
Computers that answered	the same 5	the same 5

10.1 Programs That Are Never Updated

Many people install a program once and never update it. Everything that decides credit lives on the website, so an old copy cannot pay out wrongly; what was missing was knowing which build is which. Each build now carries its commit and the commit's time in Unix seconds, sent in the User-Agent and in every status report. Once a day the program checks for a newer build and shows a notice that never blocks local use. Two website settings name the oldest build that still earns and a mode: in *warn* mode older programs are flagged and their owners emailed once; in *enforce* mode they earn nothing, while sign-in, balances and local use are never affected. For WorkPacks, every build with an identity is credited only through commit-then-random-audit, and a modified copy that hides its identity looks like an old build, so a modified program has no third path.

10.2 Tests

Every test is written up step by step, with results, in `docs/TESTING.md` in the repository; Table 19 lists the most important ones.

Table 19: Main automated and live tests (automated unless stated).

What	Where	Result
Harness, real model	CI, Windows, macOS, Linux	23 of 23 checks per platform: download, hash, question, streamed answer
Maths-pack kernels (stage 1)	EPYC server; five builds	program 7 of 7, message format 4 of 4, browser 33 of 33 (same hashes over 200 shapes); `-workpack-check` on every build
Broker and batches (stage 3)	local Workers runtime, headless browser, rented server	broker 29 of 29, device code 7 of 7, end to end 8 of 8 (liars banned, a killed SherpaHub kept its work); prompt batches 20 of 20
Commit and random audit	website code, test database	23 of 23: honest leases pass, a wrong pack is caught, late or missing commit earns nothing without a strike, three strikes hold, GPU sum-tree proofs checked
Grid jobs	website code; live	11 of 11 and 16 of 16 (double-check agree and disagree, queueing, priority); live in Table 18
Engine sandbox	all five CI builds	16 of 16 on each, nothing skipped; chat, pictures and video then made through the sandboxed engines
Signed builds	the server agent's check	7 of 7: real signatures pass; another build, one changed bit, another key, empty signatures fail; RFC 8032 vector
Build identity and minimum for earning	website, program, Rust tests; live	17 of 17, 9 of 9 and 26 of 26: old builds credited, flagged or unpaid by mode; new builds and browsers unaffected; live copies report the new build and pass audits

11 Related Work

Table 20: PlanetCompute versus related systems, from public descriptions. ✓ present, ✗ absent, ∼ partial or historical; for PlanetCompute, ∼ means designed (and, where stated, simulated) but not yet deployed.

System	Zero-install	Native overnight	Same results, both hosts	No native job code	GPU	Frame-par. AI packs	Race + consolation	Network-distinct q.	Validated credit	Consent by design
BOINC (Anderson, 2020)	✗	✓	✗	✗	✓	∼	✗	∼	✓	✓
Folding@home (Beberg et al., 2009)	∼	✓	✗	✗	✓	✗	∼	✗	∼	✓
Gridcoin (Gridcoin contributors, 2026)	✗	✓	✗	✗	✓	✗	✗	✗	✓	✓
Golem (Golem Factory, 2016)	✗	✓	✗	∼	✓	∼	✗	✗	∼	✓
Salad (Salad Technologies, 2026)	✗	✓	✗	∼	✓	∼	✗	✗	∼	✓
Petals (Borzunov et al., 2023)	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓
Pando (Lavoie et al., 2019)	✓	✗	✗	✓	✗	✗	✗	✗	✗	✓
Gray computing (Pan et al., 2015)	✓	✗	✗	✓	✗	✗	✗	✗	✗	∼
Coinhive (2017–19)	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗
PlanetCompute	✓	✓	✓	✓	✓	∼	∼	∼	✓	✓

Table 20 summarises the comparison. BOINC (Anderson, 2004, 2020) is the closest prior work and the source of PlanetCompute's validation vocabulary, but ships native executables per platform with its account's privileges, has no browser host, and can require distinct users or hosts per quorum, not distinct networks. Gray computing (Pan et al., 2015) and Pando (Lavoie et al., 2019) computed in browsers with JavaScript, and WebAssembly remains slower than native code (Jangda et al., 2019); Golem (Golem Factory, 2016) and Salad (Salad Technologies, 2026) sell consumer compute through VMs or containers; Petals (Borzunov et al., 2023) and Ryabinin and Gusev (2020) serve or train large models across peers. Backup tasks (Dean and Ghemawat, 2004) and split-frame rendering have precedents; no surveyed system combines these ideas for AI inference on untrusted consumer machines with Freivalds-checked maths packs.

12 Open Challenges and Known Gaps

(1) **Demand:** parallel, small-input, deadline-tolerant, non-confidential jobs. (2) **Mobile:** iOS forbids long background compute. (3) **Sybil resistance** beyond account binding without intrusive identity checks. (4) **GPU determinism:** tolerances must be re-measured in each

Table 21: Hypotheses, planned experiments and the path from simulation to a real test, with status on 31 October 2026.

	Hypothesis or test	**Status**
H1	Browser WebAssembly reaches half of native throughput on the CPU kernels	int8 maths kernel on one AMD EPYC core: 24.7 against 57.9 billion operations/s (0.43): not met on one core; rest open
H2, H3	Background tabs lose over half their throughput on a major browser, the tray program nothing; at $\gamma = 0.2$ the foreground latency penalty is below 5%	Open
H4	The two hosts give bit-identical results	Met for the built-in kernels (every build); third-party modules wait for the byte ABI
H5	Browser WebGPU is within 30% of native wgpu	Waits for desktop GPU WorkPacks (planned)
H6	Frame-parallel packs cut time to video 10× with 50 machines	Open (Figure 2 is a design)
H7	Frames from different GPUs match at SSIM ≥ 0.98	Partly: NVIDIA types 0.986–0.998 (SDXL); RTX 4090 vs T4 0.987–0.992 (SD 1.5), 0.941–0.956 (FLUX)
E4, E8	Validation and the race, live, with injected cheaters	Simulated (Tables 8, 10); live runs open
E6	Energy per validated unit per host (RAPL, powermetrics)	Open; Table 17 is assumed
E9	Orchestrator: (1) simulation; (2) the same classes as Durable Objects against up to 10^6 devices generated inside Cloudflare, plus 5,000–20,000 real WebSockets; (3) real programs on varied machines and GPUs; (4) a soft launch	(1) done (Tables 11–13); (2)–(4) next

runtime and for AMD, Intel, Apple and other NVIDIA GPUs. (5) **Not yet built**, beyond Table 21: a live rerun of the split job with several packs in flight (built and simulated), stage 4 maths packs, SherpaHubs on volunteers' computers, GPU leases on real WebGPU hardware, and strikes for programs whose grid answers disagree. The Windows and Mac downloads are not yet code-signed, Metal fails on the test Mac's virtual GPU, and the native contributor runs CPU WorkPacks only.

13 Conclusion

PlanetCompute is designed to turn idle consumer machines into one compute grid for AI inference, with every credit redeemable as free processing time on it; so far it has run only on the operator's servers. It cuts the work, not the model; specifies GPU inference audits whose offline cost is a few percent; lets browsers with no model compute exactly checked maths packs (shown in a test on a local broker; the two live split jobs ran between program copies); shows in simulation that racing is safe with spare replicas, network-distinct quorums and retroactive clawback, with a hybrid rule that is both fast and fair; and, at a million simulated devices with 2% ordinary cheaters, delivered no uncorrected wrong answer in the final designs. Live tests with real volunteers, the audits inside the program and the experiments of Table 21 come next. The same WorkPacks run in a browser tab and in `planetCompute`, one program that both contributes a machine and serves models on it.

Code and Resources

All the code, scripts and test cases are in the project's GitHub repository:

https://github.com/syncaissa/planetCompute.cpp

References

David P. Anderson. BOINC: A system for public-resource computing and storage. In *Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing*, pages 4–10, 2004.

David P. Anderson. BOINC: A platform for volunteer computing. *Journal of Grid Computing*, 18(1):99–122, 2020.

Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. SliceGPT: Compress large language models by deleting rows and columns. In *International Conference on Learning Representations (ICLR)*, 2024a.

Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.

Adam L. Beberg, Daniel L. Ensign, Guha Jayachandran, Siraj Khaliq, and Vijay S. Pande. Folding@home: Lessons from eight years of volunteer distributed computing. In *IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*, 2009.

Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative inference and fine-tuning of large models. In *Proceedings of the 61st Annual Meeting of the ACL (System Demonstrations)*, pages 558–568, 2023.

Cloudflare. Durable objects. https://developers.cloudflare.com/durable-objects/, 2026.

Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. In *6th Symposium on Operating Systems Design and Implementation (OSDI)*, pages 137–150, 2004.

Shayan Eskandari, Andreas Leoutsarakos, Troy Mursch, and Jeremy Clark. A first look at browser-based cryptojacking. In *IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 58–66, 2018.

Rūsiņš Freivalds. Probabilistic machines can use less running time. In *Information Processing 77, Proceedings of the IFIP Congress*, pages 839–842, 1977.

Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2023. MIT License.

gfx-rs contributors. wgpu: Cross-platform, safe, pure-rust graphics API. https://wgpu.rs, 2026.

Golem Factory. The Golem project: Crowdfunding whitepaper. https://golem.network, 2016.

Philippe Golle and Ilya Mironov. Uncheatable distributed computations. In *Topics in Cryptology – CT-RSA 2001*, volume 2020 of *LNCS*, pages 425–440, 2001.

Gridcoin contributors. Gridcoin: Rewarding volunteer distributed computing. https://gridcoin.us, 2026.

Zhewei Huang, Tianyuan Zhang, Wen Heng, Boxin Shi, and Shuchang Zhou. Real-time intermediate flow estimation for video frame interpolation. In *European Conference on Computer Vision (ECCV)*, 2022.

Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. Not so fast: Analyzing the performance of WebAssembly vs. native code. In *USENIX Annual Technical Conference (ATC)*, pages 107–120, 2019.

Radhesh Krishnan Konoth, Emanuele Vineti, Veelasha Moonsamy, Martina Lindorfer, Christopher Kruegel, Herbert Bos, and Giovanni Vigna. MineSweeper: An in-depth look into drive-by cryptocurrency mining and its defense. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1714–1730, 2018.

Erick Lavoie, Laurie Hendren, Frédéric Desprez, and Miguel Correia. Pando: Personal volunteer computing in browsers. In *Proceedings of the 20th International Middleware Conference*, pages 96–109, 2019.

leejet and contributors. stable-diffusion.cpp: Diffusion model inference in pure C/C++. https://github.com/leejet/stable-diffusion.cpp, 2026. Release master-945-a1ded76, commit a1ded76.

Arjen K. Lenstra and Mark S. Manasse. Factoring by electronic mail. In *Advances in Cryptology: EUROCRYPT '89*, volume 434 of *Lecture Notes in Computer Science*, pages 355–371. Springer, 1990.

Michael J. Litzkow, Miron Livny, and Matt W. Mutka. Condor: A hunter of idle workstations. In *Proceedings of the 8th International Conference on Distributed Computing Systems (ICDCS)*, pages 104–111, 1988.

Yao Pan, Jules White, Yu Sun, and Jeff Gray. Gray computing: An analysis of computing with background JavaScript tasks. In *IEEE/ACM 37th International Conference on Software Engineering (ICSE)*, pages 167–177, 2015.

Max Ryabinin and Anton Gusev. Towards crowdsourced training of large neural networks using decentralized mixture-of-experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Salad Technologies. SaladCloud: Distributed GPU cloud on consumer hardware. https://salad.com, 2026.

Luis F. G. Sarmenta. Sabotage-tolerance mechanisms for volunteer computing systems. *Future Generation Computer Systems*, 18(4):561–572, 2002.

John F. Shoch and Jon A. Hupp. The "worm" programs: Early experience with a distributed computation. *Communications of the ACM*, 25(3):172–180, 1982.

Tauri Programme within The Commons Conservancy. Tauri 2: Build smaller, faster, and more secure desktop and mobile applications. https://v2.tauri.app, 2024.

W3C GPU for the Web Working Group. WebGPU. W3C Candidate Recommendation, https://www.w3.org/TR/webgpu/, 2025.

W3C WebAssembly Working Group. WebAssembly core specification. https://webassembly.github.io/spec/core/, 2025.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4): 600–612, 2004.

Maxwell I. Zimmerman, Justin R. Porter, Michael D. Ward, et al. SARS-CoV-2 simulations go exascale to predict dramatic spike opening and cryptic pockets across the proteome. *Nature Chemistry*, 13:651–659, 2021.

Syncaissa Systems Inc. • PlanetCompute.ai • syncaissa@outlook.com
https://github.com/syncaissa/planetCompute.cpp